

Here's a **complete explanation of error handling and flow** in your project using custom classes and middleware:

Overall Error Handling Flow:

1. User sends a request (`GET` , `POST` , `PUT` , etc.)
2. Middleware like `validateListing` checks the request.
3. Route logic (async) runs via `wrapAsync` .
4. If any error occurs:
 - It's either **thrown directly** (e.g., `throw new ExpressError(...)`)
 - Or **caught by** `wrapAsync` **and forwarded to** `next(err)`
5. Finally, the error reaches the **global error-handling middleware** (`app.use((err, req, res, next) => {...})`), which displays a friendly error page.

File-Wise Workflow

1. `ExpressError.js`

Custom class for making meaningful error objects with:

```
js                                                                    Copy Edit
new ExpressError(400, "Custom error message");
```

Adds `statusCode` to the error so you can render appropriate messages in the UI.

2. `wrapAsync.js`

Utility to handle async errors:

```
js                                                                    Copy Edit
module.exports = (fn) => {
  return (req, res, next) => {
    fn(req, res, next).catch(next);
  };
};
```

➡ Wraps your route logic so any **rejected promise** (e.g., DB errors, `await` failures) are passed to the error handler.

3. `schema.js` + `validateListing` **Middleware**

Validates incoming request body using Joi **before hitting the DB**:

```
js                                                                    Copy Edit
const { error } = listingSchema.validate(req.body);
if (error) throw new ExpressError(400, error);
```

➡ Prevents bad data from being processed or saved.

💡 How Errors Are Caught (Route-Level):

✅ Joi error in middleware:

```
js                                                                    Copy Edit
app.post("/listings", validateListing, ...);
```

- If `validateListing` finds issues, it throws an `ExpressError`.
- That error goes to the final middleware (`app.use((err, ...))`).

✅ Async errors (like DB failures):

```
js                                                                    Copy Edit
app.get("/listings/:id", wrapAsync(async (req, res) => {
  const listing = await Listing.findById(id); // could fail
}));
```

- If `Listing.findById` fails, error is caught by `wrapAsync(...).catch(next)`.
- Then passed to global error middleware.

🔧 Global Error Handler

At the bottom of your `app.js`:

js

```
app.use((err, req, res, next) => {  
  let { statusCode = 500, message = "Something went wrong" } = err;  
  res.render("error.ejs", { statusCode, message });  
});
```

Copy Edit

✓ Handles:

- Validation errors
- DB errors
- Missing routes (`app.all("*", ...)`)
- Any uncaught exception

🕒 Special Cases Handled

Case	What Happens
<code>POST /listings</code> with invalid body	Joi throws error → caught → custom error page shown
<code>GET /listings/:id</code> with invalid/missing ID	Mongoose error → caught by <code>wrapAsync</code> → error page
No route matched	<code>app.all("*")</code> throws 404 → handled by global middleware

✓ Summary

Your error handling is:

1. Basic Idea

When something goes wrong (invalid input, DB error, page not found), we **don't crash the server** — instead, we **catch the error and show a nice error page**.

2. 3 Main Tools Handling Errors

Tool	Purpose
<code>ExpressError</code>	Custom error with a status code (like 404 or 400)
<code>wrapAsync</code>	Catches errors from async functions like DB calls
<code>validateListing</code>	Checks if user input is valid using Joi

3. Request Flow with Error Handling

Example: Creating a new listing

js

 Copy  Edit

```
app.post("/listings", validateListing, wrapAsync(async (req, res) => {  
  const newListing = new Listing(req.body.listing);  
  await newListing.save(); // could fail  
  res.redirect("/listings");  
}));
```

☒ Here's what happens in order:

1. `validateListing` checks data using Joi
 - ❌ If invalid → throws `ExpressError(400, message)`
 - ✅ If valid → goes to next step
2. `wrapAsync` handles the async route
 - ❌ If `save()` fails → catches error and passes it to Express
3. `app.use((err, req, res, next))` catches all errors
 - ✅ Renders `error.ejs` with the error message and status code

4. Other Error Situations

Situation	What Happens
No route matches (wrong URL)	<code>app.all("")</code> throws <code>ExpressError(404, "Page Not Found")</code>
Async DB error	<code>wrapAsync</code> catches and forwards it
Joi validation fails	<code>validateListing</code> throws <code>ExpressError(400, error.details...)</code>

5. Final Step — Global Error Handler

This middleware handles everything:

```
js
app.use((err, req, res, next) => {
  const { statusCode = 500, message = "Something went wrong" } = err;
  res.render("error.ejs", { statusCode, message });
});
```

It's like saying:

"Whatever error happened, catch it here and show a friendly message instead of crashing."

In Simple Words:

- Middleware checks the request before DB
- Async wrapper catches DB errors
- Global error handler shows error page
- `ExpressError` gives custom messages + status codes